

COMPSCI 1XC3: DEVELOPMENT BASICS

Assignment 2 - Sudoku Solver Program

Gabriel Syriani

400176049

Contents

1	Introduction	2
2	Brute-Forcing a Sudoku	2
3	Building the Brute-Force Algorithm in C	2
3.1	displaySudoku()	2
3.1.1	Code Breakdown	3
3.2	isNumberValid()	3
3.2.1	Code Breakdown	4
3.3	solveSudoku()	4
3.3.1	Code Breakdown	5
4	The Final Product	6
5	Calling the Solver in main()	8

1 Introduction

Sudoku is a world-famous puzzle where the player's objective is to fill a given 9 x 9 grid with numbers ranging from 1 to 9 (inclusive) such that no row, column or 3 x 3 sub-grid contains a duplicate digit.

The classical way to solve a Sudoku is to progressively fill the cells by using logic and the process of elimination while keeping in mind the main restriction where no row, column or 3 x 3 sub-grid contains more than one (1) of the same digit. An easy Soduko can quickly be completed as there are many cells with only one (1) possible digit that can be entered. However, a pre-filled grid with fewer numbers (i.e. harder difficulty) will require more diligence in entering digits in cells.

2 Brute-Forcing a Sudoku

However, another way to solve a Sudoku is to brute-force the solution by entering random numbers in cells until a solution is found. The optimal way to achieve this in C is to linearly fill the Soduko with random numbers while ensuring that the number being entered is not a duplicate in the current row/column/subgrid. Whenever a dead-end is encountered (i.e. the grid still has some empty cells but it is impossible to enter a number in any cell without violating the rules), the program backtracks to the previous entry and tries another number. This backtracking algorithm is repeated until a solution is found.

3 Building the Brute-Force Algorithm in C

In order to implement the backtracking idea described in Section 2, a method called "recursive-backtracking" will be using in building the solver's algorithm. Numerous functions will need to be implemented for the algorithm, as described in the subsections below.¹.

3.1 displaySudoku()

This function will be used to display the pre-filled Sudoku as well as the solution found by the algorithm. Below is the function "displaySudoku()" that will be used to print the grids. It should be noted that empty cells are noted by "0".

```
void displaySudoku(int grid[9][9]) {  
    for (int row = 0; row < 9; row++) {  
        for (int col = 0; col < 9; col++) {  
            printf("%d ", grid[row][col]);  
        }  
    }  
}
```

¹Course material taught in COMPSCI 1XC3: Development Basics, as well as ChatGPT, were used in the writing of the codes.

```

        if (((col + 1) % 3) == 0)
            printf("| ");
    }
    printf("\n");
    if (((row + 1) % 3) == 0)
        printf("-----\n");
    }
    printf("\n");
}

```

3.1.1 Code Breakdown

The for loops are used to direct the computer to iterate over every cell of the grid, printing the integer in that cell as it does so. The lines that print the vertical bar (—) and the horizontal bar (————) are included for formatting purposes, adding a vertical bar every three (3) cells, and a horizontal bar after each row of the 3 x 3 sub-grids.

3.2 isNumberValid()

During the recursive-backtracking process, the computer will be trying random numbers to fill the empty cells, and will stop once no number can be added without violating the rules of Sudoku. This function will allow the computer to only try numbers that are valid, as it would be futile to guess a number that immediately violates the rules.

```

int isNumberValid(int num, int row, int col, int grid[9][9]) {

    for (int i = 0; i < 9; i++) {
        if ((grid[row][i] == num) || (grid[i][col] == num))
            return 0;
    }

    int gridRow = row - row % 3;
    int gridCol = col - col % 3;

    for (int i = gridRow; i < (gridRow + 3); i++) {
        for (int j = gridCol; j < (gridCol + 3); j++) {
            if (grid[i][j] == num)
                return 0;
        }
    }

    return 1;
}

```

3.2.1 Code Breakdown

The function takes as inputs:

- i. The number from 1 to 9 that is being checked as a valid entry;
- ii. The row index of the cell the number is being entered into;
- iii. The column index of the cell the number is being entered into; and
- iv. The Sudoku grid being solved.

The first for-loop iterates through the row and column of the cell and returns 0 if it finds a number equal to the one being checked. The integers `gridRow` and `gridCol` are initialized to calculate the first cell of the 3 x 3 sub-grid that the cell is currently located in. This is done so that the subsequent for-loops can iterate through that sub-grid and look for the presence of a duplicate, returning 0 (not valid) if it finds one. If no duplicate is found after these checks, the function will return 1 (valid) and the computer will know to proceed with filling the cell with the number and continuing the brute-force.

3.3 solveSudoku()

Using the two (2) functions implemented above, the recursive-backtracking part of the program can now be implemented under `solveSudoku()`. The code for this function is given below:

```
int solveSudoku(int grid[9][9]) {  
  
    int isEmpty = 0;  
    int row;  
    int col;  
  
    for (row = 0; row < 9; row++) {  
        for (col = 0; col < 9; col++) {  
            if (grid[row][col] == 0) {  
                isEmpty = 1;  
                break;  
            }  
        }  
        if (isEmpty == 1)  
            break;  
    }  
  
    if (isEmpty == 0)  
        return 1;  
  
    for (int num = 1; num <= 9; num++) {  
        if (isNumberValid(num, row, col, grid)) {
```

```
        grid[row][col] = num;
        iterations++;

        if (solveSudoku(grid) == 1)
            return 1;
        grid[row][col] = 0;
    }
}

return 0;
}
```

3.3.1 Code Breakdown

This function only takes as input the given Sudoku. The first set of nested for-loops serve the purpose of iterating through the current grid and finding an empty cell (empty cell = "0"). Once an empty cell is found, the for-loops break, leaving the integers "num" and "col" with the value of the row and column indexes of the empty cell. If, after iterating through the entire grid, no empty cell is found (i.e. isEmpty is still 0), then the grid has been filled and the program will return 1 for success.

The second for-loop iterates through the integers 1 to 9 (inclusive) for the purpose of trying a random number in the cell found above. For each number being iterated, the function isNumberValid() is called to check if the entry of such number is valid in the current cell, and proceeds with entering it by reassigning the value of the current cell to that number. Once the cell is filled, the function solveSudoku() is called recursively (inside an if-statement) to continue filling the grid from its current state. The if-statement is implemented to check if the computer did indeed manage to solve the Sudoku from the current guess it made, and returns 1 for success. Otherwise, if that guess didn't lead to a solution, the computer reassigns the cell in which the guess was made to empty ("0") and tries again with the next integer from 1 to 9.

The code keeps track of the number of iterations it took to find a solution by adding 1 after each iteration (iterations++). Once every possible guess has been made and the computer still has not found a solution, the function solveSudoku() returns 0 for unsuccessful completion.

4 The Final Product

By putting all three functions together and including the necessary packages, the algorithm below is attained. All that remains is the code inside the main function to utilize the solver.

```
#include <stdio.h>

int N = 9;
int iterations = 0;

void displaySudoku(int grid[9][9]) {
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            printf("%d ", grid[row][col]);
            if (((col + 1) % 3) == 0)
                printf("| ");
        }
        printf("\n");
        if (((row + 1) % 3) == 0)
            printf("-----\n");
    }
    printf("\n");
}

int isNumberValid(int num, int row, int col, int grid[9][9]) {
    for (int i = 0; i < 9; i++) {
        if ((grid[row][i] == num) || (grid[i][col] == num))
            return 0;
    }

    int gridRow = row - row % 3;
    int gridCol = col - col % 3;

    for (int i = gridRow; i < (gridRow + 3); i++) {
        for (int j = gridCol; j < (gridCol + 3); j++) {
            if (grid[i][j] == num)
                return 0;
        }
    }

    return 1;
}
```

```
int solveSudoku(int grid[9][9]) {  
  
    int isEmpty = 0;  
    int row;  
    int col;  
  
    for (row = 0; row < 9; row++) {  
        for (col = 0; col < 9; col++) {  
            if (grid[row][col] == 0) {  
                isEmpty = 1;  
                break;  
            }  
        }  
        if (isEmpty == 1)  
            break;  
    }  
  
    if (isEmpty == 0)  
        return 1;  
  
    for (int num = 1; num <= 9; num++) {  
        if (isNumberValid(num, row, col, grid)) {  
            grid[row][col] = num;  
            iterations++;  
  
            if (solveSudoku(grid) == 1)  
                return 1;  
            grid[row][col] = 0;  
        }  
    }  
  
    return 0;  
}
```

5 Calling the Solver in main()

For the purpose of this program, the input of the given Sudoku will be hard-coded into the main function. The main function will call `solveSudoku()` and will print the solution using `displaySudoku()` if it finds one, as well as the number of iterations it took to find it. Otherwise, it will print "No solution exists." if a solution is not found. The main function can look something like this:

```
int main() {

    int grid[9][9] = {
        {0, 2, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 0, 6, 0, 0, 0, 0, 3},
        {0, 7, 4, 0, 8, 0, 0, 0, 0},
        {0, 0, 0, 0, 0, 3, 0, 0, 2},
        {0, 8, 0, 0, 4, 0, 0, 1, 0},
        {6, 0, 0, 5, 0, 0, 0, 0, 0},
        {0, 0, 0, 0, 1, 0, 7, 8, 0},
        {5, 0, 0, 0, 0, 9, 0, 0, 0},
        {0, 0, 0, 0, 0, 0, 0, 4, 0}};

    printf("The input Sudoku puzzle:\n");
    displaySudoku(grid);

    if (solveSudoku(grid)) {
        printf("Solution found after %d iterations:\n", iterations);
        displaySudoku(grid);
    }

    else
        printf("No Solution exists.\n");
}
```